



Instituto Tecnológico de Mazatlán



**Sistema de Control Remoto por Voz usando Arduino, WiFi y
Reconocimiento de Voz**

Carrera:

Ingeniería en Sistemas Computacionales

Materia:

Sistemas Programables

Profesor:

Miguel Hernandez Barrón

Alumnos:

Rosa Guadalupe Romero Navarro

Javier Ramón Quintero Valdez

Gabriel Alejandro Barrón Sarabia

Sistema de Control Remoto por Voz usando Arduino, WiFi y Reconocimiento de Voz

1. Objetivo

Diseñar e implementar un sistema de control remoto para un carrito robótico, utilizando reconocimiento de voz desde un dispositivo móvil como interfaz de mando. El objetivo es que el usuario, mediante comandos hablados (avanzar, retroceder, detener y variar la velocidad), controle de forma inalámbrica el movimiento de un motor DC a través de una placa Arduino conectada a una red WiFi.

2. Introducción

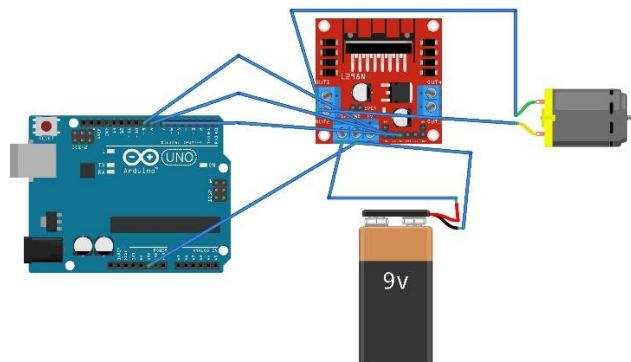
Este proyecto combina electrónica, programación embebida y desarrollo de aplicaciones móviles para crear un sistema de control por voz. La aplicación móvil (desarrollada en MIT App Inventor) captura el audio del usuario, lo convierte a texto mediante el bloque de reconocimiento de voz y envía una petición HTTP a la dirección IP del Arduino, que interpreta el comando recibido y acciona el motor a través de un puente H (L298N).

3. Materiales

- Placa Arduino UNO R4 WiFi (con librería WiFiS3.h)
- Módulo controlador de motor L298N
- Motor DC con llanta
- Fuente de alimentación de 9V para el motor
- Cables de conexión (dupont)
- Dispositivo móvil con la aplicación desarrollada en MIT App Inventor
- Red WiFi local para la comunicación entre el celular y el Arduino

4. Diagrama del circuito

El diagrama muestra la conexión entre el Arduino, el módulo L298N y el motor DC, alimentado por una batería de 9V independiente del Arduino:



Arduino	L298N
Pin 9 (PWM)	ENA
Pin 8	IN1
Pin 7	IN2
GND	GND

Las salidas OUT1 y OUT2 del L298N se conectan a las dos terminales del motor DC. La alimentación de potencia (batería de 9V) se conecta a las entradas +12V y GND del módulo, de forma independiente a la alimentación lógica del Arduino.

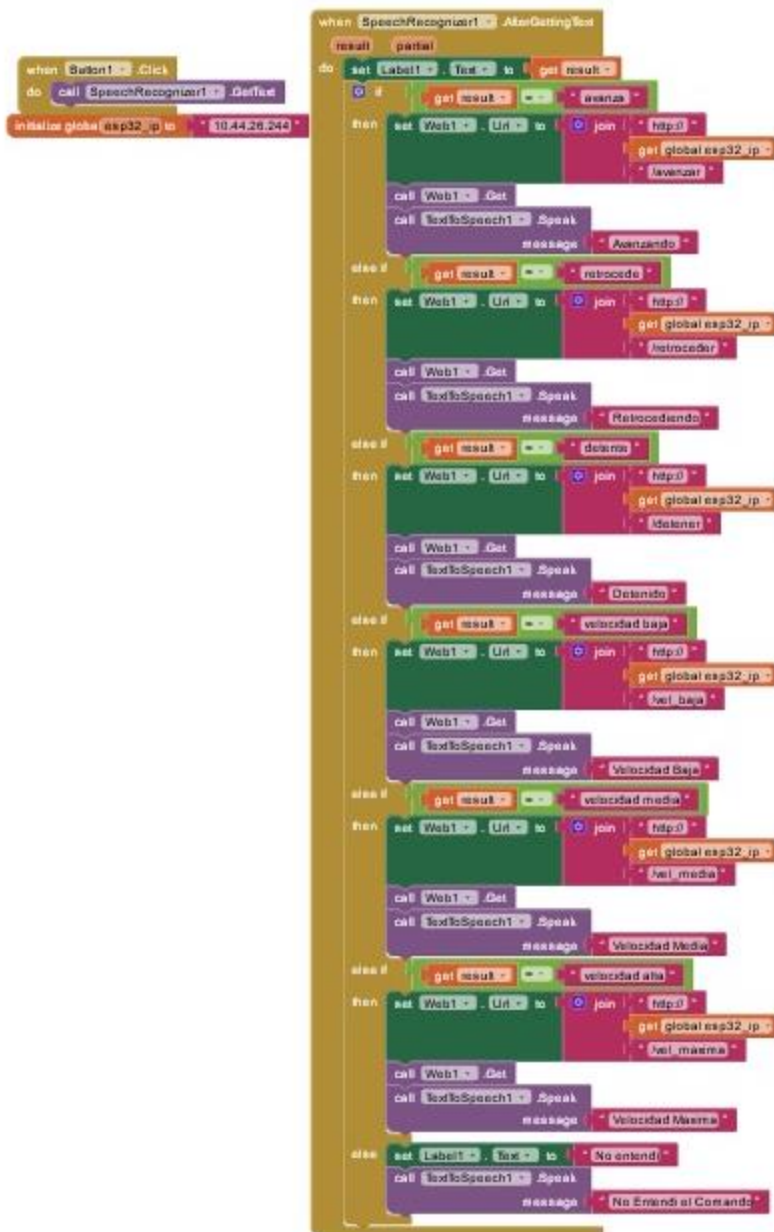
5. Armado físico

A continuación, se muestra el circuito armado de forma física, integrando el Arduino, el módulo controlador de motor y la fuente de alimentación. A diferencia del diagrama de referencia (que usa una batería de 9V), en el montaje real se utilizó una fuente de poder de PC (ATX) para alimentar el motor a través del L298N, aprovechando su riel de 12V, que ofrece corriente más estable que una pila de 9V:



6. Aplicación móvil (MIT App Inventor)

La aplicación utiliza un bloque de reconocimiento de voz (SpeechRecognizer) que, al terminar de escuchar, evalúa el texto obtenido y compara con las palabras clave: 'avanzar', 'retroceder', 'detener', 'velocidad baja', 'velocidad media' y 'velocidad alta'. Según el resultado, arma una URL con la IP del Arduino (por ejemplo [http://\[IP\]/avanzar](http://[IP]/avanzar)) y realiza una petición web (componente Web1.Get). Además, utiliza el bloque TextToSpeech para confirmar por voz al usuario la acción ejecutada.



7. Código del Arduino

El siguiente código corre en el Arduino: se conecta a la red WiFi, levanta un servidor en el puerto 80 y, al recibir una petición, identifica el comando en la URL para mover el motor o ajustar su velocidad.

```
#include <WiFiS3.h>
```

```
const char* ssid = "Motorola";
const char* password = "w2vrday";
```

```

WiFiServer server(80);
int status = WL_IDLE_STATUS;

// Pines según tu diagrama
const int ENA = 9; // PWM - velocidad
const int IN1 = 8; // Dirección
const int IN2 = 7; // Dirección

int velocidadActual = 150;

// 0 = detenido, 1 = avanzando, -1 = retrocediendo
int direccionActual = 0;

void setup() {
  pinMode(ENA, OUTPUT);
  pinMode(IN1, OUTPUT);
  pinMode(IN2, OUTPUT);
  detener();

  Serial.begin(115200);
  while (!Serial);

  while (status != WL_CONNECTED) {
    Serial.println("Conectando...");
    status = WiFi.begin(ssid, password);
    delay(5000);
  }

  Serial.println("Conectado!");
  Serial.print("IP: ");
  Serial.println(WiFi.localIP());
  Serial.println("Esperando comandos de voz...");
  Serial.println("-----");

  server.begin();
}

void avanzar() {
  direccionActual = 1;
  digitalWrite(IN1, HIGH);
  digitalWrite(IN2, LOW);
  analogWrite(ENA, velocidadActual);
}

void retroceder() {
  direccionActual = -1;
  digitalWrite(IN1, LOW);
  digitalWrite(IN2, HIGH);
  analogWrite(ENA, velocidadActual);
}

void detener() {
  direccionActual = 0;
  digitalWrite(IN1, LOW);
  digitalWrite(IN2, LOW);
  analogWrite(ENA, 0);
}

void actualizarVelocidad() {
  if (direccionActual == 1) {
    avanzar();
  } else if (direccionActual == -1) {
    retroceder();
  }
}

```

```

}

void loop() {
  WiFiClient client = server.available();

  if (client) {
    String request = "";
    while (client.connected() && client.available()) {
      char c = client.read();
      request += c;
      if (c == '\n') break;
    }

    String respuesta = "OK";

    // Confirmación clara en el Monitor Serie
    Serial.println("-----");
    Serial.print(">> Mensaje recibido: ");
    Serial.println(request);

    if (request.indexOf("GET /avanzar") >= 0) {
      avanzar();
      respuesta = "Avanzando";
      Serial.println(">> Comando reconocido: AVANZAR");
    } else if (request.indexOf("GET /retroceder") >= 0) {
      retroceder();
      respuesta = "Retrocediendo";
      Serial.println(">> Comando reconocido: RETROCEDER");
    } else if (request.indexOf("GET /detener") >= 0) {
      detener();
      respuesta = "Detenido";
      Serial.println(">> Comando reconocido: DETENER");
    } else if (request.indexOf("GET /vel_baja") >= 0) {
      velocidadActual = 100;
      actualizarVelocidad();
      respuesta = "Velocidad baja";
      Serial.println(">> Comando reconocido: VELOCIDAD BAJA (100)");
    } else if (request.indexOf("GET /vel_media") >= 0) {
      velocidadActual = 180;
      actualizarVelocidad();
      respuesta = "Velocidad media";
      Serial.println(">> Comando reconocido: VELOCIDAD MEDIA (180)");
    } else if (request.indexOf("GET /vel_maxima") >= 0) {
      velocidadActual = 255;
      actualizarVelocidad();
      respuesta = "Velocidad maxima";
      Serial.println(">> Comando reconocido: VELOCIDAD MAXIMA (255)");
    } else {
      respuesta = "Comando desconocido";
      Serial.println(">> ADVERTENCIA: comando NO reconocido");
    }

    Serial.print(">> Enviando respuesta a la app: ");
    Serial.println(respuesta);
    Serial.println("-----");

    client.println("HTTP/1.1 200 OK");
    client.println("Content-Type: text/plain");
    client.println();
    client.println(respuesta);

    delay(1);
    client.stop();
  }
}

```

8. Funcionamiento / Procedimiento

- El Arduino se conecta a la red WiFi configurada (ssid y password) y obtiene una dirección IP local.
- Esa IP se ingresa en el bloque 'initialize global esp32_ip' de la aplicación móvil.
- El usuario presiona el botón y da un comando de voz (por ejemplo, 'avanzar').
- La app reconoce el texto, arma la URL correspondiente y la envía por WiFi al Arduino.
- El Arduino recibe la petición, identifica el comando en el texto (ej. 'GET /avanzar') y activa el motor en la dirección y velocidad correspondiente.
- El Arduino responde a la app con un mensaje de confirmación, que se reproduce como voz mediante TextToSpeech.

9. Resultados

Se logró establecer la comunicación WiFi entre la aplicación móvil y el Arduino de forma estable. Los comandos de voz fueron reconocidos correctamente y el motor respondió a las órdenes de avanzar, retroceder, detener y cambio de velocidad (baja, media y máxima). El monitor serie del Arduino permitió verificar en tiempo real cada comando recibido y la respuesta enviada a la aplicación.

10. Conclusión

El proyecto permitió integrar conceptos de redes (comunicación HTTP sobre WiFi), electrónica de potencia (control de motores mediante un puente H) y desarrollo de aplicaciones móviles con reconocimiento de voz. El resultado es un sistema funcional de control remoto por voz, que puede ampliarse a futuro agregando más motores para direccionamiento, sensores de distancia o retroalimentación de estado en la aplicación.